

ARL Debugging Challenge Report

Yassin Ibrahim Ali

Summary

9 distinct issues were found across the 3 files: 1 build-configuration error, 1 missing standard-library and 7 logic errors. Each is detailed below with its symptom, root cause, fix, and the underlying concept.

Output after all fixes:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\user> cd C:\Users\user\Desktop\ARL-2027-Yassin-Mission\Debugging_Assignment
PS C:\Users\user\Desktop\ARL-2027-Yassin-Mission\Debugging_Assignment> .\build\rover_safety_monitor.exe
ARL Rover Safety Monitor
Valid detections: 3
Nearest obstacle: cone
World position: (96.0, 62.0)
Stopping distance: 20.0 m
Emergency brake: ENGAGE
PS C:\Users\user\Desktop\ARL-2027-Yassin-Mission\Debugging_Assignment>
```

Issue 1: Missing source file in CMakeLists

Symptom: The project failed to link: the linker reported undefined references for every function declared in `safety_monitor.hpp`.

Root Cause: The call only listed `main.cpp` as a source file. The file containing the actual function bodies, `src/safety_monitor.cpp`, was never listed.

```
add_executable(rover_safety_monitor
    main.cpp
    src/safety_monitor.cpp
)
```

Reasoning: The build system still has to be told which files to actually compile and hand to the linker. Every source file with definitions referenced elsewhere in the project must be listed as a build target's source.

Issue 2: Missing library

Symptom: Compilation failed with: 'optional' in namespace 'std' does not name a template type, pointing at the declaration of findNearestObstacle.

Root Cause: The header uses `std::optional<Obstacle>` as a return type, but `<optional>` is not included and is not guaranteed to be pulled in transitively by other standard headers.

```
#include <string>
#include <vector>
#include <optional> // included the library
```

Reasoning: Each standard library facility used directly in a file should be explicitly included in that file.

Issue 3: Never converting from degrees to radians

Symptom: World coordinates for every obstacle were wrong.

Root Cause: `cos` and `sin` both expect radians, so the trig functions were being evaluated on a raw degree value as if it were already in radians.

```
std::vector<Obstacle> obstacles;
const double pi = std::acos(-1.0); // declaring pi
const double headingRadians = pose.headingDegrees * (pi / 180.0); // convert to radians
const double cosine = std::cos(headingRadians);
const double sine = std::sin(headingRadians);
```

Reasoning: The standard degrees-to-radians conversion is $\text{radians} = \text{degrees} \times (\pi/180)$. $\text{acos}(-1.0)$ is an accurate way to obtain π without relying on a compiler-specific macro (see the M_PI note at the end of this report for why that matters in practice).

Issue 4: Sign error in the world-X coordinate transform

Symptom: After fixing the radian conversion, world X coordinate did not match expectations.

Root Cause: Converting a detection from the rover's local (forward, left) frame into world coordinates is a 2D rotation by the heading angle. "Left" is the direction 90° counter-clockwise from "forward". For a rotation by θ , the correct combination is $\text{worldX} = \text{originX} + \text{forward} \cdot \cos\theta - \text{left} \cdot \sin\theta$ and $\text{worldY} = \text{originY} + \text{forward} \cdot \sin\theta + \text{left} \cdot \cos\theta$. The supplied code added the $\text{left} \cdot \sin\theta$ term instead of subtracting it, which is correct for Y but incorrect for X.

```
detection.left,
pose.worldX + cosine * detection.forward - sine * detection.left, // corrected sign
pose.worldY + sine * detection.forward + cosine * detection.left,
range,
```

Reasoning: This is the standard 2D rotation matrix applied to a local-frame offset vector to express it in world coordinates.

Issue 5: Loop bound skips the last detection

Symptom: The reported “Valid detections” count was one lower than expected, and in general the very last detection in the input list was never evaluated at all, regardless of whether it was valid.

Root Cause: The loop condition was `index + 1 < detections.size()` instead of `index < detections.size()`.

```
for (std::size_t index = 0; index < detections.size(); ++index) { // corrected index
    const auto& detection = detections[index];
    const double range = std::hypot(detection.forward, detection.left);
```

Reasoning: The bound for iterating every valid index of a zero-indexed container of size N is `index < N`, covering indices 0 through N-1 inclusive. Any variant that shifts this boundary silently drops elements from one end.

Issue 6: Confidence validity check uses the wrong range

Symptom: Detections were being accepted or rejected inconsistently with the spec.

Root Cause: The specification requires confidence to be between the configured minimum and 1.0, inclusive. The code instead checked that confidence was between 0.0 and the configured minimum — the opposite range.

```
const double range = std::hypot(detection.forward, detection.left);
const bool validConfidence = detection.confidence >= config.minimumConfidence
    && detection.confidence <= 1.0;
const bool validRange = range > 0.0 && range <= config.maximumRangeMeters;
```

Reasoning: A range check must be validated directly against the written specification rather than assumed from the variable's name.

Issue 7: findNearestObstacle selects the farthest obstacle, not the nearest

Symptom: The function reported the obstacle with the greatest range as the “nearest” obstacle — the exact opposite of the intended behaviour.

Root Cause: The comparison inside the loop was `obstacle.range > nearest->range`, which keeps replacing the candidate every time a farther obstacle is found, converging on the maximum range in the list rather than the minimum.

```
for (const auto& obstacle : obstacles) {  
    if (obstacle.range < nearest->range) { // corrected comparison operator  
        nearest = &obstacle;  
    }  
}
```

Reasoning: A minimum search must replace the current best candidate only when a smaller value is found.

Issue 8: Speed never converted from km/h to m/s

Symptom: The computed stopping distance was far larger than expected (an order of magnitude off from the specification's example requires).

Root Cause: `speedMps` was assigned `speedKph` directly, with no unit conversion.

```
double calculateStoppingDistance(double speedKph, const SafetyConfig& config) {  
    const double speedMps = speedKph * (5.0/18.0); // unit conversion  
    const double reactionDistance = speedMps * config.reactionTimeSeconds;  
    const double brakingDistance = speedMps * speedMps
```

Reasoning: $1 \text{ km/h} = 5/18 \text{ m/s}$.

Issue 9: shouldEmergencyBrake was an unimplemented stub

Symptom: The function always returned false, so the program always reported “Emergency brake: clear” regardless of input.

Root Cause: The function body was simply return false; with no logic implemented at all.

```
bool shouldEmergencyBrake(const std::vector<Obstacle>& obstacles, double speedKph, const SafetyConfig& config) {  
    const double stoppingDistance = calculateStoppingDistance(speedKph, config);  
  
    for (const auto& obstacle : obstacles) {  
        if (obstacle.forward >= 0.0 && std::abs(obstacle.left) <= config.laneHalfWidthMeters && obstacle.forward <= stoppingDistance) {  
            return true;  
        }  
    }  
  
    return false;  
} // fixed the whole brake function instead of returning false every time
```

Reasoning: This is an existential quantifier over a collection, which maps naturally onto a loop that returns true the moment a qualifying element is found, and falls through to a single return false only after the entire collection has been checked without success. The three sub-conditions must be combined with logical AND, since an obstacle only counts as a braking trigger if all three hold simultaneously.

Additional Note: M_PI Portability Issue

This was not one of the graded logic bugs in the supplied code, but a genuine build issue encountered while implementing the fix for Issue 3 (degrees-to-radians conversion), and is included here for completeness and honesty about the debugging process.

The first fix attempted for the radian conversion used the M_PI macro from <cmath>:

```
const double headingRadians =  
    pose.headingDegrees * (M_PI / 180.0);
```

compilation failed with:

```
error: 'M_PI' was not declared in this scope
```

Fix

Replaced the macro with a computed constant using a standard library function that is guaranteed portable:

```
const double pi = std::acos(-1.0);  
const double headingRadians =  
    pose.headingDegrees * (pi / 180.0);
```

How the Corrected System Processes a Detection

Traced below using the “cone” detection from the supplied telemetry in main.cpp.

1. Validity check, All three numeric fields are finite. Range -- valid range, Confidence -- valid confidence. The detection passes all checks and proceeds to coordinate conversion.

2. Coordinate transform Heading 90° is converted to radians: $90 \times (\pi/180) = \pi/2$. $\cos(\pi/2) = 0$, $\sin(\pi/2) = 1$. World X = 96. World Y = 62. The obstacle is recorded with world position (96.0, 62.0), matching the specification's expected output exactly.

3. Nearest-obstacle selection, Among the three obstacles that pass validity checks, the smallest range belongs to "cone", so it is selected and reported as the nearest obstacle.

4. Stopping distance, Speed 36 km/h converts to 10 m/s. Reaction distance = 10 m. Braking distance = 10 m. Stopping distance = 20 m.

5. Emergency brake decision, The loop checks each valid obstacle in turn. The "barrier" obstacle (forward = 18.0, left = 1.0) is checked first: $\text{forward} \geq 0$ (true), $|\text{left}| \leq 1.50$ (true), $\text{forward} \leq \text{stopping distance } 20.0$ (true). All three conditions hold simultaneously, so the function returns true and the program reports "Emergency brake: ENGAGE", without needing to examine the remaining obstacles.

References

- AI tool used: Claude was used throughout this assignment. It did not write the submitted fixes; instead, I derived, wrote, and typed each fix myself, which Claude then checked against the specification's expected output and traced through by hand to confirm correctness. It also helped troubleshoot the local Windows build environment (Python/CMake/compiler

installation and the M_PI portability issue documented above) and helped structure this report.